PSet 2: Language modeling with transformers

Complete each `# TODO` or `...` blank and answer the written-response prompts. Run the cells in order.

In [1]:
```
# external library imports
# install all of these dependencies before starting
import torch
from tokenizers import Tokenizer
```

Part I: Tokenizing, embedding, and un-embedding · 25 points

Written language consists of strings of characters, but neural networks operates on vectors. The first step of language modeling is to convert strings of text into a sequence of fixed-size vectors. We first split up a continuous string of text into discrete chunks called **tokens** and assign each of these an integer ID. Different models vary in the details of how they chunk text and which specific IDs they assign tokens, so we just load a pre-existing tokenizer to play with:

Setup: load the tokenizer

This cell assumes the notebook's working directory the parent folder `PSET2`, which also contains some pre-trained model components we will use.

In [2]:
```
from pathlib import Path

weights_dir = Path("weights/TinyStories-1Layer-21M")
tokenizer = Tokenizer.from_file(str(weights_dir / "tokenizer.json"))
V = tokenizer.get_vocab_size()
print(f"Vocabulary size V = {V:,}")
```

```
Vocabulary size V = 50,257
```

Exercise 1: tokenize a sentence and then decode the token IDs back to text · 3 points

Use `tokenizer.encode(text).ids` to obtain a Python list of token IDs, and `tokenizer.decode(ids)` to convert a list of IDs back to a string.

Complete the cell below. The loop displays the text represented by each individual token using `repr`, which makes spaces visible.

In [3]:
```python
sentence = "Lily found a little red ball."

# TODO: encode the sentence, then convert the list of IDs to a torch.long te
token_ids = tokenizer.encode(sentence).ids

print("Token IDs:", token_ids)
print("Number of tokens:", len(token_ids))

for token_id in token_ids:
    print(token_id, repr(tokenizer.decode([token_id])))

# TODO: decode the entire sequence of IDs.
decoded_sentence = tokenizer.decode(token_ids)
print("Decoded sentence:", repr(decoded_sentence))
assert decoded_sentence == sentence
```

```
Token IDs: [43, 813, 1043, 257, 1310, 2266, 2613, 13]
Number of tokens: 8
43 'L'
813 'ily'
1043 ' found'
257 ' a'
1310 ' little'
2266 ' red'
2613 ' ball'
13 '.'
Decoded sentence: 'Lily found a little red ball.'
```

Written response · 3 points: How many tokens does this sentence have? Does each token correspond to one word? Give an example from the output. Then try changing `Lily` to `lily` or adding a space before the sentence, and describe how the tokenization changes.

The sentence has 8 tokens. The tokens don't always correspond to words. For example, "Lily" was split into "L" and "ily". However, the other words were preserved. The tokenization remained the same for all except "lily" and " lily" after changing the input. Only the first token changed, being "L", "l", and " l" for "Lily", "lily", and " lily" respectively.

Why use dense embeddings?

Token IDs tell us which vocabulary items are present, but they do not provide a useful geometry for neural computation. One option is a **one-hot vector** with one coordinate per vocabulary item: all coordinates are zero except the coordinate for that token. For this vocabulary, each such vector would have 50,257 coordinates, and every pair of distinct tokens would be equally far apart.

Instead, we use a learned **embedding matrix** $E \in \mathbb{R}^{V \times D}$, where V is the vocabulary size and $D = 1024$ is the embedding dimension. Row i is a dense, real-valued vector representing token i. Training adjusts these vectors so that the model can use them to

predict text. Their dot products and distances can reflect useful relationships; individual coordinates need not have simple interpretations.

For a token ID i, the embedding is $E[i]$. This lookup is equivalent to multiplying a one-hot row vector by E, without constructing the large one-hot vector. For a sentence with T tokens, indexing all its IDs produces a matrix $X \in \mathbb{R}^{T \times D}$.

Setup: load the embedding matrix

Again, this cell assumes the notebook's working directory the parent folder `PSET2`.

In [4]:
```
weights_dir = Path("weights/TinyStories-1Layer-21M")

if not (weights_dir / "pytorch_model.bin").is_file():
    raise FileNotFoundError("Cannot find the downloaded TinyStories weights.

checkpoint = torch.load(weights_dir / "pytorch_model.bin", map_location="cpu

# E has one embedding vector per vocabulary item (one row per token).
E = checkpoint["transformer.wte.weight"].detach()
# U maps a row vector in embedding space to a row of vocabulary scores.
U = E.T
del checkpoint
V, D = E.shape
print(f"Vocabulary size V = {V:,}; embedding dimension D = {D:,}")
print("Embedding matrix E:", tuple(E.shape))
print("Un-embedding matrix U:", tuple(U.shape))
```

```
Vocabulary size V = 50,257; embedding dimension D = 1,024
Embedding matrix E: (50257, 1024)
Un-embedding matrix U: (1024, 50257)
```

Exercise 2: look up token embeddings · 3 points

Use tensor indexing to obtain an embedding for every token in your sentence. Do not loop over tokens or construct one-hot vectors.

In [5]:
```
# TODO: select the rows of E corresponding to token_ids.
embeddings = E[torch.tensor(token_ids)]
print("Token ID shape:", len(token_ids))
print("Embedding shape:", tuple(embeddings.shape))
assert embeddings.shape == (len(token_ids), D)

# Inspect a few coordinates of the first token's vector.
print("First token:", repr(tokenizer.decode([token_ids[0]])))
print("First eight embedding coordinates for first token:")
print("> ", embeddings[0, :8])
```

```
Token ID shape: 8
Embedding shape: (8, 1024)
First token: 'L'
First eight embedding coordinates for first token:
>  tensor([ 0.0627, -0.0039, -0.0544,  0.0234, -0.0680, -0.0192, -0.0058,
-0.0031])
```

Written response · 2 points: If the same token appears twice in a sentence, will its two embedding vectors be identical at this stage? Can these vectors alone distinguish the token's position or surrounding words? Explain briefly. Later sections will add position embeddings and attention to the model.

The embedding vectors would be identical. No operation is applied to the embedding vector as it is extracted from the matrix, so it directly corresponds to the token. So, they cannot distinguish position and context.

From vectors to vocabulary scores: un-embedding

The language model eventually needs one score for each possible output token. **Un-embedding** maps a vector in the model's hidden space to these vocabulary scores, called *logits*. Here we just use the transpose of its embedding matrix, $U = E^\top \in \mathbb{R}^{D \times V}$:

$$S = XU, \qquad S \in \mathbb{R}^{T \times V}.$$

The score $S_{t,j}$ is the dot product of vector X_t with the embedding of vocabulary token j . These scores are not probabilities. Applying softmax along the vocabulary dimension converts them to probabilities, but you can select the highest-scoring token directly with `argmax(dim=-1)` .

Un-embedding is not an inverse of embedding. In general, $EE^\top$ is not the identity matrix. Another token's vector can have a larger dot product with a given embedding than that embedding has with itself, so embedding and then un-embedding need not recover the original token. Decoding token IDs, by contrast, does not use embeddings at all.

In the full language model, un-embedding is applied to hidden vectors *after* position embeddings, attention, the feed-forward network, and final normalization. Those scores predict the **next** token. Here, we apply it directly to input embeddings to understand the operation in isolation.

Exercise 3: un-embed and choose tokens · 4 points

Complete the matrix multiplication and choose one token ID per row. Use `@` for matrix multiplication and take the maximum over the vocabulary dimension.

In [6]:
```python
# TODO: map embeddings to vocabulary scores with U.
scores = embeddings @ U
assert scores.shape == (len(token_ids), V)

# TODO: choose the highest-scoring token at each position.
recovered_ids = torch.argmax(scores, dim=-1)
assert len(recovered_ids) == len(token_ids)

for original, recovered in zip(token_ids, recovered_ids):
    print(repr(tokenizer.decode([original])), "->", repr(tokenizer.decode([r

print("Fraction of IDs recovered:", (recovered_ids == torch.tensor(token_ids
```

```
'L' -> 'L'
'ily' -> 'ily'
' found' -> ' found'
' a' -> ' a'
' little' -> ' little'
' red' -> ' red'
' ball' -> ' ball'
'.' -> '.'
Fraction of IDs recovered: 1.0
```

Written response · 3 points: What are the shapes of `embeddings`, `U`, and `scores`, and why are they compatible for matrix multiplication? Did this sentence recover every token? Does that result guarantee recovery for other sentences?

`embeddings` has shape (8, 1024), `U` has shape (1024, 50257). Since the number of columns in `embeddings` is the same as the number of rows in `U`, matrix multiplication to `scores` is well-defined and gives shape (8, 50257). This sentence recovered every token, but that does not guarantee the same for other sentences. As the text said earlier, $EE^T \neq I$. Another embedding may have a higher dot product than an embedding with itself, making it possible for a different token to be recovered.

Exercise 4: put the entire sequence together · 4 points

Write a function that takes a sentence and performs all five steps:

1. Tokenize the sentence into a tensor of integer IDs.
2. Look up the embedding vector for each token.
3. Un-embed the vectors into vocabulary scores.
4. Choose the highest-scoring token at each position.
5. Decode the resulting IDs into a string.

Use the provided `tokenizer`, `E`, and `U`. The function should work for any nonempty sentence, without depending on variables from the earlier exercises. Return the decoded string and both ID tensors so you can check what changed. Note that `tokenizer.decode` requires a python list as input, so you may need to convert tensors back to lists.

In [7]:
```python
def embedding_round_trip(text):
    # TODO: tokenize, embed, un-embed, choose IDs, and decode.
    input_ids = tokenizer.encode(text).ids
    vectors = E[torch.tensor(input_ids)]
    vocabulary_scores = vectors @ U
    output_ids = torch.argmax(vocabulary_scores, dim=-1)
    output_text = tokenizer.decode(output_ids.tolist())
    return output_text, input_ids, output_ids


for text in ["Lily found a little red ball.", "Once upon a time, there was a
    output_text, input_ids, output_ids = embedding_round_trip(text)
    print("Original: ", repr(text))
    print("Recovered:", repr(output_text))
    print("All IDs match:", torch.equal(torch.as_tensor(input_ids), torch.as
    print()

# TODO: try a sentence of your own and inspect the result.
my_sentence = "At some points I realized I was swinging my arms so I tried t
output_text, input_ids, output_ids = embedding_round_trip(my_sentence)
print("Original: ", repr(my_sentence))
print("Recovered:", repr(output_text))
print("All IDs match:", torch.equal(torch.as_tensor(input_ids), torch.as_ten
```

```
Original:  'Lily found a little red ball.'
Recovered: 'Lily found a little red ball.'
All IDs match: True

Original:  'Once upon a time, there was a small dragon.'
Recovered: 'Once upon a time, there was a small dragon.'
All IDs match: True

Original:  'At some points I realized I was swinging my arms so I tried to n
ot.'
Recovered: 'At some points I realized I was swinging my arms so I tried to n
ot.'
All IDs match: True
```

Written response · 3 points: Describe what each step in your function does. Which steps use learned weights? If a recovered sentence differs from the original, which step selected a different token? If your examples all match, explain why that still does not make un-embedding a mathematical inverse.

First, the tokenizer converts the input text into a list of token ids. Next, the ids are mapped to their corresponding vectors using the embedding matrix. Next, these vectors are multiplied with the un-embedding matrix (transpose of embedding matrix). The resulting product is used as the input of an argmax function, which finds the token each vectorized embedding has the highest dot product with. These tokens are then decoded as the output text.

The only learned weights are in the embedding matrix (and consequently, the un-embedding matrix). The tokenizer is just a pre-determined mapping of text to ids. So, learned weights are used in steps 2 (tokens to embeddings) and 3 (embeddings @ un-

embeddings).

All the examples match, but if a sentence did differ, it would have been at the fourth step where argmax was applied to the un-embedding scores.

The un-embedding matrix is simply the transpose of the embedding matrix. The transpose of a matrix is the matrix's inverse for an orthogonal matrix, meaning the vectors are all orthonormal. This is not true in general for the embedding matrix, so it is not necessarily a mathematical inverse.

Part II: Attention mechanism · 45 points

In Part I, each token received an embedding determined only by its ID. How can a token's representation incorporate information from other tokens in the sentence? **Self-attention** computes a weighted average of information from other positions, with weights that depend on the tokens themselves.

We will build a single attention head using the TinyStories tokenizer and embedding matrix from Part I. To make the behavior easy to interpret, we will provide handcrafted query and key projections rather than use the model's trained projections. We will also leave out position embeddings for now.

Queries, keys, and values

Let $X \in \mathbb{R}^{T \times D}$ contain the embeddings of a sentence with T tokens. We apply three linear maps:

$$q = XW_Q, \qquad k = XW_K, \qquad v = XW_V.$$

Here $W_Q, W_K \in \mathbb{R}^{D \times d_k}$ and $W_V \in \mathbb{R}^{D \times d_v}$. Each position therefore has three vectors:

- A **query** describes what information this position seeks.
- A **key** describes what information a position makes available for matching.
- A **value** contains the information that will be passed along if that position is attended to.

These are useful interpretations of learned vectors, rather than literal questions or labels in a trained model. The projections are shared across positions. This is *self*-attention because all three sets of vectors come from the same input sequence.

For a query at position i, define a score, a normalized attention weight, and an output:

$$s_{ij} = q_i \cdot k_j, \qquad a_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell=1}^{T} \exp(s_{i\ell})}, \qquad z_i = \sum_{j=1}^{T} a_{ij} v_j.$$

A larger dot product gives a position more weight. Softmax makes the weights positive

and ensures that they sum to one. For all queries at once, this becomes

$$A = \mathrm{softmax}\left(qk^\top\right), \qquad Z = Av,$$

where softmax is applied across the **key positions in each row**. A has shape (T, T), while Z has shape (T, d_v).

Causal attention: A model predicting the next token must not access future positions. It masks scores for $j > i$ before softmax. We omit that mask in these examples to focus on the matching operation. This is an illustration of attention, not a complete next-token predictor.

Setup: a head that distinguishes animals from colors

Our first sentence is:

> I like blue. I like dog. My favorite animal is...

The important words here have single tokens when preceded by a space. We will construct two-dimensional keys with the following meanings:

Token text (including its preceding space)	Key vector
`red` , `blue`	$[1, 0]$
`cat` , `dog`	$[0, 1]$
Other tokens in our examples	$[0, 0]$

The query for `animal` is $[0, 4]$, and the query for `color` is $[4, 0]$. Thus the animal query matches animal keys, and the color query matches color keys. The factor of four makes the matching tokens stand out after softmax; it is a teaching choice, not a pretrained model parameter. Without it, these two-dimensional scores would produce fairly diffuse attention.

We set $W_V = I_D$, so values are just the original embedding vectors. In a trained attention head, the value projection is also learned and generally is not the identity.

Run the next cell as provided. It creates linear maps with these behaviors on the tokens used in our examples. The pseudoinverse is simply a way to fit those prescribed input/output pairs; you do not need to derive it for this exercise. These maps are not guaranteed to have interpretable behavior on other vocabulary tokens.

We use `W_V` for the value projection to avoid confusing it with the vocabulary size `V` from Part I.

In [8]:
```
import matplotlib.pyplot as plt
```

```
animal_sentence = "I like blue. I like dog. My favorite animal is..."
color_sentence = "I like blue. I like dog. My favorite color is..."
update_sentence = "I like blue. Actually, now I like red. My favorite color

word_ids = {}
for word in ["red", "blue", "cat", "dog", "animal", "color"]:
    ids = tokenizer.encode(" " + word).ids
    assert len(ids) == 1, f"Expected one token for {word!r}"
    word_ids[word] = ids[0]

# Include all example tokens so that background queries and keys are zero.
example_ids = sorted(set(
    word_ids.values()
) | {token_id for text in [animal_sentence, color_sentence, update_sentence]
     for token_id in tokenizer.encode(text).ids})
fit_X = E[example_ids].double()
target_q = torch.zeros(len(example_ids), 2, dtype=torch.float64)
target_k = torch.zeros_like(target_q)
for row, token_id in enumerate(example_ids):
    if token_id in [word_ids["red"], word_ids["blue"]]:
        target_k[row] = torch.tensor([1.0, 0.0])
    elif token_id in [word_ids["cat"], word_ids["dog"]]:
        target_k[row] = torch.tensor([0.0, 1.0])
    if token_id == word_ids["animal"]:
        target_q[row] = torch.tensor([0.0, 4.0])
    elif token_id == word_ids["color"]:
        target_q[row] = torch.tensor([4.0, 0.0])

# Solve fit_X @ W = target for each supplied projection.
W_Q = (torch.linalg.pinv(fit_X) @ target_q).float()
W_K = (torch.linalg.pinv(fit_X) @ target_k).float()
W_V = torch.eye(D)
d_k = W_Q.shape[1]
assert d_k == 2
# The supplied maps agree with their targets up to floating-point error.
torch.testing.assert_close(E[example_ids] @ W_Q, target_q.float(), atol=1e-5
torch.testing.assert_close(E[example_ids] @ W_K, target_k.float(), atol=1e-5
print("Projection shapes:", tuple(W_Q.shape), tuple(W_K.shape), tuple(W_V.sh
```

```
Projection shapes: (1024, 2) (1024, 2) (1024, 1024)
```

Exercise 1: attend to the animal · 12 points

Translate the equations above into PyTorch operations. First tokenize and embed `animal_sentence`, then construct q, k, and v with matrix multiplication. Select the query at the position containing `animal`.

Compute that query's score for every key, apply softmax across positions, and plot the resulting attention weights. Finally, compute the weighted sum of the values. Use matrix multiplication rather than a loop for the attention computation.

In [9]:
```
# TODO: obtain token embeddings and their query, key, and value vectors.
attention_ids = torch.tensor(tokenizer.encode(animal_sentence).ids)
X = E[attention_ids]
```

```
q = X @ W_Q
k = X @ W_K
v = X @ W_V
animal_position = (attention_ids == word_ids["animal"]).nonzero(as_tuple=Tru

# TODO: compare the selected query against all keys and normalize.
animal_scores = q[animal_position] @ k.T
animal_weights = torch.softmax(animal_scores, dim=-1)
animal_output = animal_weights @ v

assert q.shape == k.shape == (len(attention_ids), 2)
assert v.shape == X.shape == (len(attention_ids), D)
assert animal_output.shape == (D,)
torch.testing.assert_close(animal_weights.sum(), torch.tensor(1.0))
print("Animal query:", q[animal_position].round(decimals=3))
for token_id, score, weight in zip(attention_ids.tolist(), animal_scores, an
    print(f"{tokenizer.decode([token_id])!r:14s} score={score.item():6.3f}
```

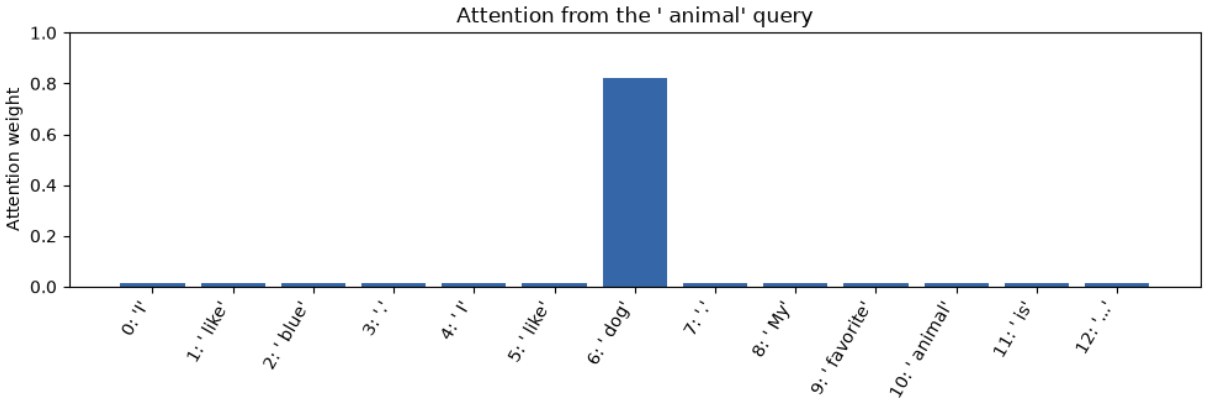
```
Animal query: tensor([0., 4.])
'I'             score=-0.000  weight=0.0150
' like'         score=-0.000  weight=0.0150
' blue'         score= 0.000  weight=0.0150
'.'             score=-0.000  weight=0.0150
' I'            score=-0.000  weight=0.0150
' like'         score=-0.000  weight=0.0150
' dog'          score= 4.000  weight=0.8198
'.'             score=-0.000  weight=0.0150
' My'           score=-0.000  weight=0.0150
' favorite'     score= 0.000  weight=0.0150
' animal'       score=-0.000  weight=0.0150
' is'           score=-0.000  weight=0.0150
'...'           score=-0.000  weight=0.0150
```

In [10]:

```
# Provided plotting helper: position numbers distinguish repeated tokens.
def plot_attention(ids, weights, title):
    labels = [f"{position}: {tokenizer.decode([token_id])!r}"
              for position, token_id in enumerate(ids.tolist())]
    fig, ax = plt.subplots(figsize=(10, 3.5))
    ax.bar(range(len(labels)), weights.tolist(), color="#3566a8")
    ax.set_xticks(range(len(labels)), labels, rotation=60, ha="right")
    ax.set_ylabel("Attention weight")
    ax.set_ylim(0, 1)
    ax.set_title(title)
    fig.tight_layout()
    plt.show()

plot_attention(attention_ids, animal_weights, "Attention from the ' animal'

# Inspect the output vector and compare it to the targeted token's value.
dog_position = (attention_ids == word_ids["dog"]).nonzero(as_tuple=True)[0].
print("Output shape:", tuple(animal_output.shape))
print("First eight output coordinates:", animal_output[:8])
print("First eight dog-value coordinates:", v[dog_position, :8])
print("Weight on dog:", animal_weights[dog_position].item())
```




```
Output shape: (1024,)
First eight output coordinates: tensor([ 0.0242, -0.0089, -0.0816, -0.0040,
-0.0082, -0.0268, -0.0639,  0.0426])
First eight dog-value coordinates: tensor([ 0.0249, -0.0108, -0.1014, -0.003
1, -0.0093, -0.0347, -0.0732,  0.0524])
Weight on dog: 0.8198146820068359
```

Written response · 7 points: Which token receives the most attention, and why? Do tokens with zero scores receive zero attention? Is the attention output exactly the value at the most-attended position?

' dog' receives the most attention as its key points in a similar direction to the query for "animal". Consequently, its dot product has the highest score versus other tokens, which is preserved with softmax. Tokens with zero scores don't have zero attention, as softmax forces a positive output value for a real input value. The attention output is not exactly the value at the most-attended position, but a weighted average of the value vectors.

Exercise 2: change the query to color · 8 points

Replace `animal` with `color` in the sentence. Recompute the token embeddings, queries, keys, attention weights, and value sum. The supplied projection matrices stay fixed: we are changing the input to the same attention head.

For reuse, implement a function that returns the token IDs, attention weights, and output vector for a selected query word. Then apply it to `color_sentence` and plot the weights.

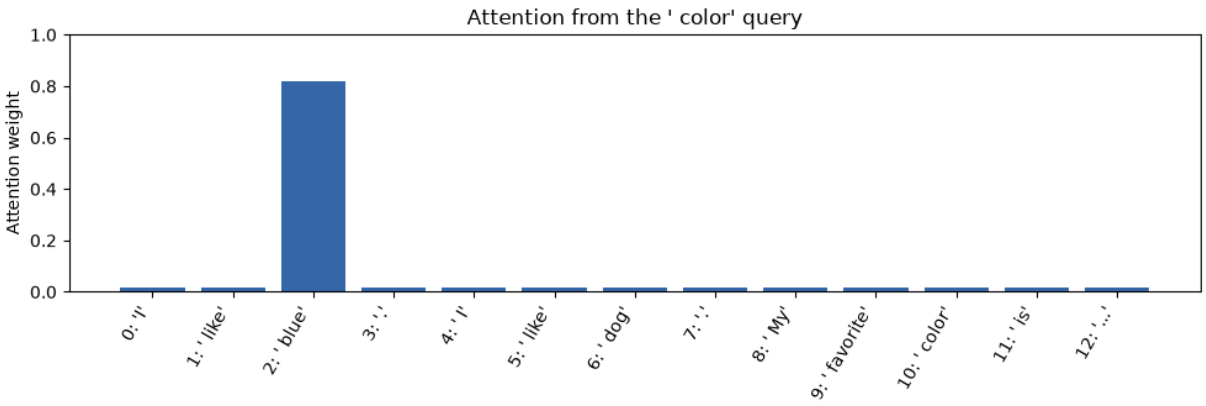
In [11]:
```python
def attend_from_word(text, query_word):
    # TODO: construct the vectors for this new sentence.
    ids = torch.tensor(tokenizer.encode(text).ids)
    X = E[ids]
    q = X @ W_Q
    k = X @ W_K
    v = X @ W_V
    query_position = (ids == word_ids[query_word]).nonzero(as_tuple=True)[0]

    # TODO: compute attention and return the value sum for that query.
```

```
    scores =  q[query_position] @ k.T
    weights = torch.softmax(scores, dim=-1)
    output = weights @ v
    return ids, weights, output

color_ids, color_weights, color_output = attend_from_word(color_sentence, "c
plot_attention(color_ids, color_weights, "Attention from the ' color' query"
blue_position = (color_ids == word_ids["blue"]).nonzero(as_tuple=True)[0].it
print("Weight on blue:", color_weights[blue_position].item())
print("First eight output coordinates:", color_output[:8])

# Both computations should select their intended categories.
assert attention_ids[animal_weights.argmax()].item() == word_ids["dog"]
assert color_ids[color_weights.argmax()].item() == word_ids["blue"]
```




```
Weight on blue: 0.819814920425415
First eight output coordinates: tensor([ 0.0637, -0.0296,  0.0120,  0.0165,
-0.0175,  0.0086,  0.0013,  0.0376])
```

Written response · 5 points: Why did the high-attention position change, even though none of the projection matrices changed? Which parts of the computation differ between the two sentences?

The query input changed, as we are now querying for tokens related to "color". This changed the score calculation step, leading to the resulting attention weights.

Exercise 3: which color is current? · 4 points

Now consider:

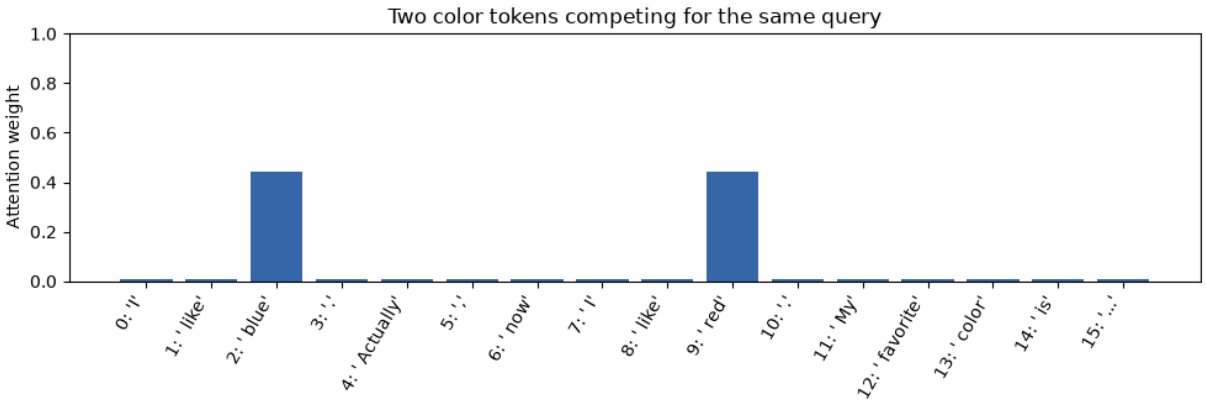
> I like blue. Actually, now I like red. My favorite color is...

As readers, we interpret the later statement as updating the earlier one. Apply the same attention head and inspect the weights on `blue` and `red`. Do they reflect that update?

In [12]:
```
# TODO: run attention on the updated sentence.
update_ids, update_weights, update_output = attend_from_word(update_sentence
plot_attention(update_ids, update_weights, "Two color tokens competing for t
blue_position = (update_ids == word_ids["blue"]).nonzero(as_tuple=True)[0].i
```

```
red_position = (update_ids == word_ids["red"]).nonzero(as_tuple=True)[0].ite
print("Weight on blue:", update_weights[blue_position].item())
print("Weight on red: ", update_weights[red_position].item())
print("Total weight on the two colors:", update_weights[[blue_position, red_
print("First eight output coordinates:", update_output[:8])

# Equal keys give equal scores and therefore equal attention weights.
torch.testing.assert_close(update_weights[blue_position], update_weights[red
```




```
Weight on blue: 0.44318050146102905
Weight on red:  0.44317975640296936
Total weight on the two colors: 0.8863602876663208
First eight output coordinates: tensor([ 0.0478, -0.0204, -0.0190,  0.0326,
-0.0004,  0.0171, -0.0267,  0.0428])
```

Written response · 9 points: Explain why the two colors receive equal weight. What does their contribution to the output vector look like? Would adding a causal mask let this query distinguish the earlier color from the later one? What additional information might help?

The colors receive equal weight because the query only looks for words related to color, not their position or context. They contribute to the output vector equally, and are the major contributors out of all tokens. A causal mask wouldn't help as the query "color" is after both "blue" and red", so the mask wouldn't hide either and the weights would remain equal. Positional information would help, as the correct answer is later in the text.

Part III: Full architecture · 28 points

We can now put these components together to build a language model. We will use the pretrained "TinyStories" model, including its learned attention and feed-forward network, rather than the handcrafted projections from Part II. This model has **one transformer block**. The block contains both an attention sublayer and a feed-forward sublayer.

What else does the model need?

Position embeddings. We add a learned vector for each position to the token

embedding. Two occurrences of the same token can then start with different representations depending on their positions. TinyStories has position embeddings for positions 0 through 2047.

Multi-head attention (MHA). Instead of using a single query/key/value projection, TinyStories has 16 attention heads, each with 64-dimensional query, key, and value vectors. Each head performs the attention operation from Part II. Their outputs are concatenated and passed through a learned output projection to produce one 1024-dimensional vector per position. Different heads can learn different ways of gathering information. A **causal mask** prevents each position from attending to later positions.

A feed-forward network (MLP). Attention mixes information across positions. The MLP transforms the vector at each position separately, using the same weights at every position. Here it expands the vector from 1024 to 4096 coordinates, applies a nonlinear activation (GELU), and maps back to 1024 coordinates. The nonlinearity lets this network do more than a single linear transformation.

Residual connections. Each sublayer computes an update that is added to its input. This allows the representation to carry information forward while also being modified by attention and the MLP.

Layer normalization (LN). Layer normalization normalizes the coordinates of each position's vector and then applies learned scale and shift parameters. This model applies it before attention, before the MLP, and once more before un-embedding. We provide these modules; your task is to put them in the right places.

With X denoting the sequence of vectors, the computation is:

$$X_0 = \text{token embeddings} + \text{position embeddings},$$

$$X_1 = X_0 + \mathrm{MHA}(\mathrm{LN}_1(X_0)),$$

$$X_2 = X_1 + \mathrm{MLP}(\mathrm{LN}_2(X_1)),$$

$$\text{logits} = \mathrm{LN}_f(X_2)E^\top.$$

Each row of the logits contains scores for the token **following** that position. To predict the next token after a prompt, select the last row and take its highest-scoring vocabulary entry.

Setup: load the remaining trained modules

Run this cell as provided. The local `tiny_stories.py` file defines the attention module and loads the checkpoint using only PyTorch. We extract its pretrained components here so you can assemble the forward pass yourself. We reuse the embedding matrix `E` and tokenizer already loaded in Part I.

All modules are set to evaluation mode. We will run inference without computing gradients.

In [13]:
```
from torch import nn
from tiny_stories import TinyStories

trained_model = TinyStories.from_pretrained(weights_dir)
position_embedding = trained_model.position_embedding
MHA = trained_model.attention
MLP = nn.Sequential(
    trained_model.expand,
    nn.GELU(approximate="tanh"),
    trained_model.contract,
).eval()
norm1 = trained_model.norm1
norm2 = trained_model.norm2
final_norm = trained_model.final_norm

# Confirm that Part I and these modules use the same checkpoint.
torch.testing.assert_close(E, trained_model.token_embedding.weight)
# The extracted modules remain available after deleting this container.
del trained_model
print("Position embeddings:", tuple(position_embedding.weight.shape))
print("Attention heads:", MHA.num_heads, "| Coordinates per head:", MHA.head
print("MLP:", MLP)
```

```
Position embeddings: (2048, 1024)
Attention heads: 16 | Coordinates per head: 64
MLP: Sequential(
  (0): Linear(in_features=1024, out_features=4096, bias=True)
  (1): GELU(approximate='tanh')
  (2): Linear(in_features=4096, out_features=1024, bias=True)
)
```

Exercise 1: assemble the forward pass · 15 points

Complete `LLM.forward` using the sequence of operations above. The constructor is provided. The model accepts a Python list of token IDs and returns the ID of the most likely next token. Tokenization and decoding happen outside the model.

Use shape `(1, T)` for the input IDs, where the first dimension represents a batch containing one prompt. Embeddings and the intermediate representations then have shape `(1, T, D)`, and vocabulary scores have shape `(1, T, V)`.

Select the scores at the **last** position to predict the next token.

In the example below the class, tokenize the prompt, call the model, and decode the predicted ID to inspect the result.

In [14]:
```
class LLM(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        self.token_embedding = nn.Embedding.from_pretrained(E)
        self.position_embedding = position_embedding
        self.MHA = MHA
        self.MLP = MLP
        self.norm1 = norm1
        self.norm2 = norm2
        self.final_norm = final_norm

    @torch.no_grad()
    def forward(self, ids):
        # TODO: create a batch containing the supplied list of IDs.
        if not 1 <= len(ids) <= self.position_embedding.num_embeddings:
            raise ValueError("The prompt must contain between 1 and 2048 tok
        token_ids = torch.tensor(ids).unsqueeze(0)

        # TODO: combine token identity with position information.
        positions = torch.arange(token_ids.shape[1]).unsqueeze(0)
        x = self.token_embedding(token_ids) + self.position_embedding(positi

        # TODO: attention and MLP each add an update to the representation.
        x = x + self.MHA(self.norm1(x))
        x = x + self.MLP(self.norm2(x))

        # TODO: normalize, un-embed, and choose from the last position.
        x = self.final_norm(x)
        logits = x @ self.token_embedding.weight.T
        next_token_id = logits[0, -1].argmax().item()
        return next_token_id


llm = LLM().eval()
prompt = "Once upon a"
prompt_ids = tokenizer.encode(prompt).ids
next_id = llm(prompt_ids)
next_token = tokenizer.decode([next_id])
print("Prompt:", repr(prompt))
print("Most likely next token:", repr(next_token))
print("Together:", prompt + next_token)
assert next_token == " time"
```

```
Prompt: 'Once upon a'
Most likely next token: ' time'
Together: Once upon a time
```

Written response · 3 points: Why do we select only the last row of logits?

As stated earlier, each row of logits scores the token following the corresponding position. We want to predict the token that comes after the whole input, which would be after the last token, and thus the last row of logits.

Exercise 2: generate a story one token at a time · 10 points

A language model can produce a longer continuation by repeatedly predicting the next token and appending its ID to the input sequence. This is **autoregressive generation**:

each new prediction is conditioned on the prompt and all the tokens generated so far.

Write a loop that calls `llm` to predict up to `n_tokens` new tokens. Tokenize the prompt once. On each iteration, pass the complete list of IDs generated so far, then append the predicted ID. Stop early if the model predicts its end-of-text ID (50256). Decode the full sequence only after the loop. Keeping IDs avoids changing token boundaries by repeatedly decoding and re-tokenizing the text. Our implementation recomputes the representations of the whole sequence each time to keep the steps easy to follow.

This procedure always picks the most likely token, which is called **greedy decoding**. Sampling is another option, but we will use greedy decoding here.

In [15]:
```python
def generate_story(prompt, n_tokens):
    # TODO: tokenize the prompt.
    ids = tokenizer.encode(prompt).ids
    for _ in range(n_tokens):
        # TODO: predict from the complete sequence and append the new ID.
        next_id = llm(ids)
        if next_id == 50256:
            break
        # TODO: append the next ID to the sequence.
        ids.append(next_id)
    # TODO: decode the completed sequence.
    return tokenizer.decode(ids)


prompt = "Once upon a"
story = generate_story(prompt, n_tokens=100)
new_tokens = len(tokenizer.encode(story).ids) - len(tokenizer.encode(prompt)
print(story)
print("\nNew tokens:", new_tokens)
assert new_tokens == 100
```

```
Once upon a time, there was a little girl named Lily. She loved to play with
her toys and her favorite toy was a teddy bear. One day, Lily's mommy told h
er that they were going to visit her grandma. Lily was excited because she l
oved her grandma very much.

When they arrived at her grandma's house, Lily saw a big box with a lock on
it. She asked her mommy, "What's inside?" Her mommy said, "It's a surprise,

New tokens: 100
```

AI reflections · 2 points

AI assistance is allowed under the course policy. In a few sentences, describe how, if at all, you used AI while completing this problem set. What was useful? What was confusing, incorrect, or unhelpful? How did you check the suggestions or outputs you used?

If you did not use AI, say so and briefly describe how you checked your work.

You receive full credit for completing this reflection, regardless of whether you used AI or found it helpful.

I didn't use AI. Since subsequent steps in the notebook require correct answers in previous cells, I can mostly verify correctness of my answers. Also, I used the pytorch documentation to ensure I used correct syntax.